

Project Details: Modeling Liquidity Dynamics in Single-Event
Polymarket Markets as a Function of Time until Market
Resolution

10/22/2025

Baenan McKeown

Programming Implementation

I am particularly interested in modeling financial markets, and prediction markets offer a novel and tractable setting for doing so. Their relative simplicity makes it feasible to model their liquidity dynamics within the scope of this course, while also allowing for future extensions beyond it. To support this work, I am undertaking a significant programming component involving the development of a C++ client that interfaces directly with the Polymarket Central-Limit Order Book (CLOB).

The CLOB provides all data necessary for this project, as it records every trade executed within a given market. For several single-event markets, I will connect to the Polymarket CLOB via a web-socket connection (which the Polymarket web server exposes to developers) implemented using C++'s Boost::beast and Boost::asio. I will then collect order data over a fixed observation period. This will constitute the primary dataset for model estimation and analysis.

The collected data will be used to validate the mathematical model described below, allowing for empirical assessment of its fit and predictive behavior.

Given my background in high-performance numerical methods, I will solve the model computationally using a numerical solver implemented in C++. A computational implementation is particularly important due to current uncertainty about the empirical forms of $I(\tau)$ and $R(\tau)$, which will only be determined after data collection and smoothing.

Liquidity Model Formulation

We consider a prediction market for n mutually exclusive outcomes, where q_i denotes the total number of outstanding shares of outcome i . The market is defined by a convex *cost function*

$$C : \mathbb{R}^n \rightarrow \mathbb{R}, \quad (1)$$

which specifies the total cost required to reach a given share vector $q = (q_1, \dots, q_n)$. A trader moving the state of the market from q to q' pays the amount

$$\Delta C = C(q') - C(q), \quad (2)$$

and receives a unit payout for the realized outcome.

Price and Liquidity in the Scalar Case ($n = 2$)

For a binary market ($n = 2$, such as a sports game), there is one independent variable $q \in \mathbb{R}$, corresponding to the net shares of one outcome. The instantaneous market price is given by the gradient of the cost function:

$$p(q) = \frac{dC}{dq}. \quad (3)$$

The *liquidity* of the market is then captured by the curvature of C :

$$\ell(q) = \frac{d^2C}{dq^2}. \quad (4)$$

A small trade Δq induces a price change

$$\Delta p \approx \ell(q) \Delta q. \quad (5)$$

Hence, a large value of $\ell(q)$ implies prices are highly sensitive to order flow (low liquidity), while a small $\ell(q)$ indicates stable prices (high liquidity).

Empirical Application: Supply–Demand Dynamics of Liquidity

A realistic description of macroeconomic liquidity treats liquidity as the outcome of competing supply and demand forces. The above

$$\ell(q) = \frac{d^2C}{dq^2}$$

is used in order to demonstrate how liquidity represents the curvature of the price of a security within a market. What follows describes how the curvature of this cost function evolves, providing insight into how liquidity evolves over time.

Let $\tau = T - t$ denote the time remaining until market resolution, and let $\ell(\tau)$ denote the instantaneous (scalar) liquidity of the market.

We model the rate of change of liquidity as

$$\frac{d\ell}{d\tau} = a I(\tau) - b R(\tau) + c \ell(\tau), \quad (6)$$

where:

- $I(\tau)$ represents *trading intensity*, or aggregate demand for liquidity (e.g., trade volume per unit time). This parameter contributes to liquidity because a higher trade volume means that there are more traders/liquidity providers who are willing to buy/sell securities, which contributes to an efficient, and thus liquid, market.
- $R(\tau)$ represents *bid-ask spread*, or the difference between the highest price for which someone is willing to buy a security, and the lowest price for which one is willing to sell that security. This parameter is negative because the presence of a bid-ask spread means that the market is inefficient, which implies that the market is illiquid.
- $a, b, c > 0$ are coefficients controlling how each factor influences the liquidity trajectory.

The first term captures the depletion of liquidity due to trading activity, the second term captures inflows of liquidity provision, and the final term represents natural withdrawal or decay of existing liquidity as the market approaches resolution.

Under constant coefficients, the solution is

$$\ell(\tau) = e^{c\tau} \left(\ell_0 + \int_0^\tau e^{-cs} [a I(s) - b R(s)] ds \right), \quad (7)$$

where ℓ_0 is the initial liquidity at some reference time. This formulation admits a wide range of dynamics depending on the functional forms of $I(\tau)$ and $R(\tau)$.

In empirical applications, $I(\tau)$ and $R(\tau)$ can be estimated by measurable quantities such as trade count, trade volume, or net liquidity inflow on the platform, which will be computed computationally. The a, b, c constants will be set once an I have an idea of the shape of the data. If the model proves to be inadequate for modelling market liquidity, this will be commented on.

Bounded Liquidity Dynamics

To ensure realistic dynamics, liquidity is assumed to be bounded within $[\ell_{\min}, \ell_{\max}]$. The rate of change of liquidity is therefore modeled as a *saturated* logistic-type process:

$$\frac{d\ell}{d\tau} = [a I(\tau) - b R(\tau)] + c \ell(\tau) \left(1 - \frac{\ell(\tau)}{\ell_{\max}} \right) - d (\ell_{\min} - \ell(\tau))_+,$$

where $a, b, c, d > 0$ are coefficients. The middle term $c \ell(1 - \ell/\ell_{\max})$ enforces an upper bound, causing growth to slow as ℓ approaches $\ell_{\max}$, while the final term penalizes values below $\ell_{\min}$ through a soft lower saturation. This specification prevents explosive or negative liquidity trajectories and reflects that market depth cannot grow indefinitely nor fall below a minimal baseline.

For moderate liquidity levels where $\ell_{\min} \ll \ell \ll \ell_{\max}$, the model reduces to the linear form

$$\frac{d\ell}{d\tau} \approx a I(\tau) - b R(\tau) + c \ell(\tau),$$

recovering the standard unsaturated dynamics.

$I(\tau)$ and $R(\tau)$

The functions $I(\tau)$ and $R(\tau)$, representing trading intensity and bid-ask spread respectively, will be computed directly from observed trade and CLOB data. Trading intensity will be defined as the rate of change of cumulative trade volume with respect to time,

$$I(\tau) = \frac{dV}{dt},$$

where $V(t)$ is the cumulative traded volume and $\tau = T - t$ denotes time to resolution. The bid-ask spread will be measured as the difference between the best ask and best bid prices,

$$R(\tau) = p_a(\tau) - p_b(\tau),$$

or, in normalized form, $R(\tau) = [p_a(\tau) - p_b(\tau)]/p_m(\tau)$, where $p_m(\tau)$ is the mid-price.

Data Collection

I will be connecting to the Polymarket Websocket, which can be accessed at

```
wss://ws-subscriptions-clob.polymarket.com/ws/
```

There are a variety of API calls available to me from here, including but not limited to:

```
curl --request GET --url https://clob.polymarket.com/book
```

(retrieves a summary of the order book for a given market, including bid-ask spread)

```
curl --request GET --url https://clob.polymarket.com/price
```

(retrieves market price for a given market)

Additionally, because I am using a websocket connection (upgraded HTTP connection), the Polymarket servers will automatically send me updates whenever there is a change to the CLOB of the market I am currently connected to. This is useful because, instead of me having to periodically ping Polymarket's HTTP endpoint to check for updates to the CLOB, I can simply wait for those updates to be sent to me.

This data will be written into a binary file from which it can be read from another C++ program, which will contain the numerical solver. I will be able to compare the solver's computed values from the actual liquidity values of the market's CLOB in this program. Python will be used for any graphical needs, as I find it's graphing libraries (I use Matplotlib) the easiest to use.

Additional Reading

My formulation of the model above was inspired by the following two papers:

- Decentralised finance and automated market making: Execution and speculation
- A General Theory of Liquidity Provisioning for Prediction Markets

They provided information regarding the methods that a market maker within a CLOB market model would use in order to estimate the liquidity of the market in which they work. They were also useful in providing context as to how market makers work in decentralized finance (DeFi) environments.

The information that I still need is less research-paper-oriented, and is primarily related to the implementation of this project. I will be extensively using C++'s Boost documentation in order to collect the data needed for this project. Additionally, because I tentatively plan on using RK4 as my time-stepping schema, I will need to review the procedure for the numerical method. These resources can be found below:

- Boost::beast
- RK4 Methods